Sistema de Gerenciamento de Alunos

Guia para a prova oral · onde está cada requisito no projeto (Haskell + IHP)

> **Resumo em uma frase:** aplicação web em Haskell com o framework IHP, com cadastro de alunos, cursos, matrículas e notas. A regra de negócio (média ponderada, situação e validações) fica isolada em **funções puras** em `Application/Domain/`, testadas sem banco, e os controllers do IHP só chamam essas funções.

Fluxo dos dados: `Schema.sql` → o IHP gera os records `Student, Course, Enrollment, Grade` → `Conversao.hs` traduz para os tipos puros → `Media.hs` / `Validacao.hs` calculam → controllers e views em `Web/` exibem.

1. Ambiente IHP + Postgres + GHC/Cabal/Nix

ONDE ESTÁ

`flake.nix`	Define o ambiente inteiro: IHP v1.6, pacotes Haskell, Postgres e o shell de desenvolvimento (devenv). Também desativa a telemetria do IHP (`IHP_TELEMETRY_DISABLED`).
`flake.lock`	Fixa a versão exata de cada dependência, então todos recebem o mesmo ambiente.
`.envrc`	Faz o direnv carregar o ambiente sozinho ao entrar na pasta.
`Makefile`, `App.cabal`, `hie.yaml`	Build do IHP. O `App.cabal` é só um stub para ferramentas (HLS); as dependências reais ficam no `flake.nix`.
`Application/Schema.sql`	Estrutura do banco Postgres. Os dados de desenvolvimento ficam em `build/db`.

O QUE DIZER

- Roda no **WSL (Ubuntu)**: o IHP depende do Nix, que não funciona nativamente no Windows.
- Versões: **GHC 9.10.3, Cabal 3.16.1, PostgreSQL 17, Nix 2.35**. O ghcup também está instalado no Ubuntu (`~/.ghcup`) para usar fora do projeto.
- `devenv up` sobe o Postgres e o servidor juntos: app em `localhost:8000`, painel do IHP em `:8001`.

Se perguntarem: *Por que Nix?* → garante builds reprodutíveis: o mesmo GHC e as mesmas bibliotecas em qualquer máquina, sem instalação manual.

2. Esqueleto do projeto com scaffolding

ONDE ESTÁ

`Main.hs`	Ponto de entrada; registra a aplicação Web.
`Config/Config.hs`	Configuração do app (porta, ambiente).
`Web/Routes.hs`	Rotas: liga cada URL a uma ação.
`Web/Types.hs`	Os controllers e as suas ações, como tipos soma.
`Web/FrontController.hs`	Registra os controllers. O layout da página fica em `Web/View/Layout.hs`.
`Application/`, `Test/`	Banco e lógica de domínio; testes.

O QUE DIZER

- O esqueleto foi criado pelo `ihp-new`: a estrutura padrão do IHP, com o padrão MVC (models gerados do banco, views em HSX, controllers).
- As rotas são **tipos**: `data StudentsController = StudentsAction | ShowStudentAction { studentId :: Id Student } ...`. Um link para uma ação inexistente nem compila.

3. Repositório Git + convenções de código

ONDE ESTÁ

`.git` **(histórico)**	Commits no padrão **Conventional Commits** (`feat: ...`).
`.gitignore`	Ignora build, `.devenv` e o segredo `Config/client_session_key.aes`.
`.stylish-haskell.yaml`	Padrão de formatação do código Haskell (imports e alinhamento).
`.github/workflows/nix-flake-check.yml`	CI: roda `nix flake check` a cada push e pull request.

`AGENTS.md`	Regras do projeto: SQL tipado, migrations com `date +%s` e testes antes de entregar.

O QUE DIZER

- Nomes: os modelos do banco seguem o IHP em inglês (`Student`); o domínio fica em português (`Aluno`, `Avaliacao`) para não colidir com os tipos gerados.
- Mudança de banco = **migration**: `Application/Migration/1791487599-add-weight-to-grades.sql` adicionou o peso das notas.
- Verificação oficial: `nix flake check --impure` (testes unitários, testes de integração com banco e build de produção).

4. Modelos (Student, Course, Enrollment, Grade) com tipos algébricos e records

ONDE ESTÁ

`Application/Schema.sql`	Tabelas `students`, `courses`, `enrollments`, `grades`. O IHP gera os records em `build/Generated/`.
`Application/Domain/Types.hs`	Versão pura: records `Aluno` (l.5), `Curso` (l.11), `Avaliacao` (l.16) e `Matricula` (l.22); tipos soma `Situacao` (l.29) e `ErroValidacao` (l.36).
`Application/Domain/Conversao.hs`	Converte os records do IHP para os do domínio (`avaliacaoDeGrade`, `matriculaDe`).

O QUE DIZER

- **Record = tipo produto** com campos nomeados (aluno **e** curso **e** semestre **e** notas).
- **Tipo soma** = uma entre várias opções (Aprovado **ou** Recuperação **ou** ...). O compilador avisa se algum caso não for tratado.
- `ErroValidacao` carrega dados em cada construtor: `NotaForaDoIntervalo Double`.

```
data Avaliacao = Avaliacao { descricao :: Text, nota :: Double, peso :: Double }

data Situacao = Aprovado | Recuperacao | Reprovado | SemNotas
    deriving (Eq, Show, Enum, Bounded)
```

Se perguntarem: *Por que dois conjuntos de tipos?* → os do IHP dependem do banco; os do domínio são valores puros e dá para testar sem IO.

5. Funções puras de domínio (IMC adaptado para média ponderada, validações)

ONDE ESTÁ

`Application/Domain/Media.hs`	`mediaPonderada` (l.35), `termosDaMedia` (l.40), `classificarCom` (l.57), `arredondar` (l.87).
`Application/Domain/Validacao.hs`	`validarNota` (l.25), `validarPeso`, `validarEmail`, `validarRa`, `validarSemestre`, `validarAvaliacao` (l.72, usa do com `Either`).
`Application/Helper/Controller.hs`	`validarCom` (l.16) adapta as validações puras para o IHP: `validateField #value (validarCom validarNota)`.
`Test/Domain/*Spec.hs`	39 testes unitários das funções puras.

O QUE DIZER

- **Analogia com o IMC:** o IMC combina duas medidas num índice e depois classifica em faixas. Aqui, notas e pesos viram uma média, Σ(nota × peso) ÷ Σ(peso), e a média é classificada: ≥ 7 Aprovado, ≥ 5 Recuperação, < 5 Reprovado.
- **Pura** = mesma entrada, mesma saída, sem efeitos (nada de banco ou IO). Por isso dá para testar isolado.
- Erros como valores: `Either ErroValidacao a` em vez de exceções. A ausência de média é `Maybe Double` (lista vazia → `Nothing`).
- Arredondamento escolar (6,625 → 6,63). O `round` do Haskell arredonda o 5 para o par mais próximo e daria 6,62.

```
termosDaMedia avaliacoes
    | somaDosPesos <= 0 = Nothing
    | otherwise = Just (somaPonderada, somaDosPesos, somaPonderada / somaDosPesos)
  where
    somaPonderada = sum [nota * peso | Avaliacao { nota, peso } <- avaliacoes]
    somaDosPesos  = sum [peso | Avaliacao { peso } <- avaliacoes]
```

6. Closures e currying aplicados

ONDE ESTÁ

`Media.hs` **l.72 e l.82**	`calculadoraComPesos` e `mediaProvasETrabalho`.
`Media.hs` **l.66**	`classificar = classificarCom corteAprovacao corteRecuperacao`.
`Validacao.hs` **l.35 e controllers**	`validarNaoVazio "nome"` é um validador pronto para o campo nome.

O QUE DIZER

- **Closure:** `calculadoraComPesos [3,3,4]` devolve uma função que *guarda* os pesos. Depois é só passar as notas de cada aluno.
- **Currying:** toda função de vários argumentos recebe um de cada vez. Os cortes vêm antes da média para permitir aplicação parcial: `classificarCom 8 6` seria uma regra de pós-graduação.
- Outros exemplos: `arredondarNota = arredondar 2`, e `zipWith (Avaliacao "")` aplica o construtor parcialmente.

```
calculadoraComPesos :: [Double] -> [Double] -> Either ErroValidacao Double
calculadoraComPesos pesos = \notas -> ...      -- a lambda "lembra" os pesos

mediaProvasETrabalho = calculadoraComPesos [3, 3, 4]   -- closure pronta
mediaProvasETrabalho [8, 6, 9]                         -- Right 7.8
```

7. List comprehensions + pattern matching

ONDE ESTÁ

`Media.hs`	Comprehensions: `termosDaMedia`, `notasAbaixoDe` (l.117), `alunosPorSituacao` (l.120), `boletim` (l.127), `contarPorSituacao` (l.134).
`Web/Controller/Enrollments.hs` **l.16**	Junta matrícula + aluno + curso + média numa única comprehension.
`Media.hs`, `Validacao.hs`	Pattern matching: `mediaSimples []`, `descreverSituacao`, `resumirAvaliacoes` (l.107), `mensagemErro` com `\case`, `validarEmail` com `case` e guards.

O QUE DIZER

- A comprehension filtra e transforma ao mesmo tempo, e o padrão do lado esquerdo do `<-` já desmonta o record: `[nomeAluno | Matricula { aluno = Aluno { nomeAluno } } <- ms]`.
- `contarPorSituacao` percorre `[minBound .. maxBound]`, ou seja, todas as situações, graças ao `Enum`/`Bounded`.
- Pattern matching no **formato** da lista: vazia, um elemento, ou vários com as-pattern (`xs@(x : _)`).

```
resumirAvaliacoes []                            = "Nenhuma nota lançada"
resumirAvaliacoes [Avaliacao { descricao, nota }] = "1 nota: " <> descricao <> ...
resumirAvaliacoes avaliacoes@(Avaliacao { nota = primeira } : _) = ...
```

8. Scaffolds gerados pelo IHP

ONDE ESTÁ

`new-controller Students` **(e Courses, Enrollments, Grades)**	Comando do IHP que lê o `Schema.sql` e gera o CRUD.
`Web/Controller/<Entidade>.hs`	7 ações: listar, novo, criar, ver, editar, atualizar, excluir.
`Web/View/<Entidade>/`	`Index, New, Show, Edit` em HSX (HTML com checagem de tipos).
`Web/Routes.hs`, `Web/Types.hs`	Um bloco `[routes| ... |]` e um tipo de controller por entidade.

O QUE DIZER

- O código gerado foi testado antes de qualquer mudança, e o teste revelou 3 falhas: nota 15 era aceita, e-mail repetido dava erro 500 e as mensagens eram genéricas.
- Depois personalizei os scaffolds: validações do domínio, checagem de duplicidade (`validarUnico`), select de aluno e de curso, e a página da matrícula com média, conta e situação.

Se perguntarem: *O que é seu e o que é gerado?* → estrutura e CRUD são do IHP; `Application/Domain/`, os testes e as validações e telas de média são implementação própria.

Demonstração ao vivo

```
cd ~/sistema-gerenciamento-alunos && devenv up     # app em http://localhost:8000
nix flake check --impure                           # testes + build de produção
```

Roteiro sugerido: **Enrollments** → **Show** → lançar uma nota com peso e ver a conta e a situação mudarem → tentar lançar nota 15 e mostrar a mensagem da validação pura.